Healthcare Notebook Cell Explanation Guide

Two-line, presentation-ready explanations for the code cells

Purpose: Use these lines while displaying each code cell so the technical steps remain connected to your official presentation flow.

Prepared for: ML model and UAT notebook walkthrough

ML Model Notebook

Use these explanations only for code cells. They follow the sequence from Gold-layer analysis through model selection, MLflow tracking and the final threshold check.

Cell 2: Imports and environment setup

Say: “Here we import all the libraries required for data preparation, model training, evaluation and MLflow tracking. We also set a fixed random seed so that our train-test split and model results are reproducible.”

Cell 3: Patient analysis by age band

Say: “Before building the model, we first analyze our Gold patient data by age band. This gives us an understanding of how patients are distributed across age groups before those attributes are used in ML.”

Output: 52 patients aged 0-17, 26 aged 18-35, 79 aged 36-55, and 26 aged 56-75.

Cell 4: Encounter analysis by age band

Say: “Here we connect the patient dimension with the encounter fact table using patient_key. This allows us to understand healthcare activity by age band and also shows how the Gold-layer star-schema relationships support analytics.”

Output: 3,483, 1,803, 5,211 and 1,808 encounters across the four age bands.

Cell 6: Feature and target selection

Say: “Here we define what information goes into our model and what we want to predict. Our five patient attributes become the features, while patient_status becomes our target.”

Say: “We have three target classes: ACTIVE, INACTIVE and PENDING, so this is a multiclass classification problem.”

Model dataset: 183 rows.

Cell 7: Train-test split, preprocessing and model training

Part A: Train-test split

Say: “First, we split our data into approximately 80% training and 20% testing. Training data teaches the model, while testing data evaluates it on unseen records.”

Say: “We also use stratification, which helps maintain the distribution of ACTIVE, INACTIVE and PENDING across both datasets.”

Part B: Imputation

Say: “Next, we handle missing categorical values using imputation. Imputation means filling missing values with a suitable replacement, and here we use the most frequently occurring category.”

Part C: One-Hot Encoding

Say: “After that, we apply One-Hot Encoding, which converts categorical values into numerical indicators the algorithms can understand. We also ignore previously unseen categories so they do not automatically break the prediction pipeline.”

Part D: Pipeline

Say: “We combine preprocessing and model training into a pipeline. This ensures exactly the same preprocessing logic is applied consistently when training and when making predictions.”

Part E: Logistic Regression and Random Forest

Say: “Then we train two classification algorithms: Logistic Regression and Random Forest. Logistic Regression models the relationship between our features and target classes, while Random Forest combines multiple decision trees.”

Say: “Both use balanced class weights to reduce bias when the target classes are not equally represented.”

Part F: Metrics

Say: “After training, both models make predictions on our test data. We calculate Accuracy, weighted Precision, Recall and F1 Score, log those results through MLflow, and then compare the models.”

Say: “Logistic Regression achieved the higher test accuracy, approximately 40.54% compared with 35.14%, so it is selected as the better-performing model of these two.”

Output: Logistic Regression - Accuracy 40.54%, Precision 38.51%, Recall 40.54%, F1 39.02%. Random Forest - Accuracy 35.14%, Precision 34.47%, Recall 35.14%, F1 34.37%.

Cell 9: Final metrics and Confusion Matrix

Say: “Here we perform the final evaluation of our selected Logistic Regression model using Accuracy, Precision, Recall and F1 Score. We also generate a Confusion Matrix, which compares the actual patient status against what the model predicted.”

Say: “The diagonal represents correct classifications, while the other cells show where the model is confusing ACTIVE, INACTIVE and PENDING.”

Metric reminder: Accuracy = overall correctness; Precision = reliability of a predicted class; Recall = actual cases identified; F1 = balance between Precision and Recall.

Cell 10: MLflow model logging

Say: “Once we finalize the model, we save it through MLflow along with its parameters, Accuracy, Precision, Recall and F1 Score. We also infer the model signature so MLflow understands the expected input and output structure.”

Say: “So MLflow gives us traceability and reproducibility, because we can identify the exact model, configuration and metrics associated with a run.”

Cell 12: Accuracy threshold and status

Say: “Here we compare the final model accuracy against our 70% expected threshold. Since our current accuracy is about 40.54%, the notebook raises a warning that model improvement is required.”

Say: “But it is treated as a warning rather than a hard failure, so the technical regression flow still completes with SUCCESS.”

UAT Notebook

These explanations cover only the code cells used for setup, reconciliation, data-quality validation and final UAT sign-off.

Cell 2: UAT setup

Say: “First, we configure the Silver and Gold locations that will be validated and initialize a failure counter. Every failed UAT check increments this counter, which is later used for final sign-off.”

Cell 4: Patient count validation

Say: “Here we compare the number of patients in Silver with the number in the Gold patient dimension. Both contain 183 patients, so the reconciliation passes.”

Say: “This confirms that we have not unexpectedly lost or added patient records while creating the Gold dimension.”

Cell 6: Encounter count validation

Say: “Here we compare the Silver encounter count against the Gold fact-table count. Both contain 12,305 encounters, so this check also passes.”

Say: “This gives us confidence that the Gold fact table preserves the expected encounter volume.”

Cell 8: Age-band validation

Say: “Here we first check the distribution of patients across the age bands and then specifically look for missing age-band values. The result shows zero missing age bands, so the validation passes.”

Say: “This is important because incomplete age-band values would affect both our Gold-layer analysis and the ML model that uses age band as a feature.”

Cell 10: Patient and encounter key validation

Say: “Here we check that our critical patient and encounter keys are populated. We found zero missing patient keys and zero missing encounter keys, so the data-quality check passes.”

Say: “These keys are important because they allow dimensions and facts to be related correctly in our Gold-layer model.”

Cell 12: Final UAT status

Say: “Finally, we check our accumulated failure count. Since there are zero failed checks, the final UAT status is SUCCESS.”

Say: “This provides our final data-level sign-off that the healthcare Gold data has passed the UAT checks defined in this notebook.”

Connection Between the Two Code Walkthroughs

ML flow: Gold analysis -> Features and target -> Preprocessing -> Train-test split -> Logistic Regression and Random Forest -> Metrics -> Confusion Matrix -> Model selection -> MLflow -> Threshold warning

UAT flow: Silver and Gold setup -> Patient reconciliation -> Encounter reconciliation -> Age-band validation -> Key-quality checks -> UAT SUCCESS

Transition to say:

“The ML notebook answers how well our model can predict patient status. The UAT notebook answers whether our healthcare data product has passed the required validation checks. Model evaluation validates predictive performance, while UAT validates the underlying data solution.”